

BreachMark on Windows 10

A step-by-step guide to installing BreachMark, running the demo, connecting a real AI model, and doing your first benchmark experiment.

For github.com/VarshSec/breachmark, version 0.1 · Written for Windows 10 (also works on Windows 11) · October 2026

Contents

1. What you need

2. Install Python

3. Get BreachMark

4. Set it up (one time)

5. Start the app and run the demo

6. A tour of the pages
7. Connect a real model

8. Your first real experiment

9. Reading the numbers

10. Command-line use

11. Troubleshooting

12. Keys, updates and resetting

1. What you need

Item	Details
Windows 10 or 11	64-bit. Any normal laptop works; BreachMark itself is small. The AI model is the heavy part and can run in the cloud.
Python 3.10 or newer	Free. Section 2 shows how to install it. Python 3.12 is a good choice.
About 300 MB of disk	Python packages plus the datasets. Local AI models (optional) need 4–20 GB more.
Internet	For installing, and for cloud models. Local models work offline once downloaded.
A model to test	Free options: a Groq or Google Gemini API key (section 7), or Ollama with a small model on your own PC.

No admin rights needed. Everything installs into your user folder. No Docker, no Node, no database server.

2. Install Python

- 1

Go to python.org/downloads and click the big download button (Python 3.12 or newer).
- 2

Run the installer. On the first screen **tick "Add python.exe to PATH"** before clicking Install Now. This is the step people miss.

- 3 Open a terminal: press `Win`, type **PowerShell**, press Enter. Then type:

```
python --version
```

You should see `Python 3.12.x` (or whichever version you installed). If you see "python is not recognized", see Troubleshooting.

3. Get BreachMark

Two ways. Pick one.

Option A: download a ZIP (no Git needed)

- 1 Open github.com/VarshSec/breachmark in a browser.
- 2 Click the green **Code** button, then **Download ZIP**.
- 3 Right-click the downloaded file, **Extract All**, and extract to a simple path such as `C:\Users\<you>\breachmark`. The extracted folder may be called `breachmark-main`; that's fine, just use that name in the commands below.

Option B: with Git

If you have Git installed (git-scm.com), in PowerShell:

```
cd $HOME  
git clone https://github.com/VarshSec/breachmark.git
```

Avoid folders with spaces or OneDrive sync if you can. `C:\Users\<you>\breachmark` is ideal. OneDrive-synced folders sometimes lock the database file while it's being written.

4. Set it up (one time)

In PowerShell, go into the folder (adjust the name if yours is `breachmark-main`):

```
cd $HOME\breachmark
```

Create a private Python environment for the project and switch to it:

```
python -m venv .venv  
.\.venv\Scripts\Activate.ps1
```

Your prompt should now start with `(.venv)`. If PowerShell refuses with a message about "execution policy", run this once and try again:

```
Set-ExecutionPolicy -Scope CurrentUser RemoteSigned
```

Install the four packages BreachMark needs:

```
pip install -r requirements.txt
```

Optional but recommended, run the self-test (about 15 seconds, no internet or model needed):

```
pip install -r requirements-dev.txt  
python -m pytest -q
```

You should see `59 passed`. That confirms the whole pipeline works on your machine.

Every time you come back later: open PowerShell, `cd $HOME\breachmark`, then `.\.venv\Scripts\Activate.ps1`. You only install once.

5. Start the app and run the demo

```
python -m breachmark
```

The first start imports the two bundled datasets (a few seconds) and prints:

```
BreachMark 0.1.0 at http://127.0.0.1:8000 (database: C:\Users\...\breachmark.db)
```

Open **`http://127.0.0.1:8000`** in any browser. Windows Firewall may ask for permission the first time; it only listens on your own PC, so either answer is safe.

On the dashboard click **Run the demo**. This creates three runs with built-in *fake* models (they're labelled "synthetic") so you can see every screen filled with data. Their scores mean nothing; they are coin flips. The demo finishes in about 10 seconds.

To stop the app, go back to PowerShell and press `Ctrl + C`. Closing the PowerShell window also stops it. Your runs and results stay in the `breachmark.db` file.

6. A tour of the pages

Page	What it's for
Dashboard	Dataset facts, the standings (best model and prompt per run), recent runs.
Dataset	Browse the 1,396 vulnerabilities: 593 from VulnSage (C/C++, Linux, Firefox, Xen) and 803 from SVEN (C/C++ and Python functions). Filter by dataset, language, CWE, project, year, size, noise. Open one to see the fix as a diff, and what every model said about it.
Runs	All runs with live progress. Open a run for its metrics, confusion matrix, per-CWE heatmap, breakdowns, every answer, report and CSV export.
Compare	Tick several runs: standings side by side, majority vote across models, detail for one run.
Playground	Paste your own code (or a public GitHub commit URL) and ask a model about it. Nothing is stored.
Prompts	Read the four built-in prompts and save your own versions.
Settings	Test connections to model providers, paste an API key for the current session, see where the database is.

7. Connect a real model

BreachMark talks to models through three "providers". Here is how to set each up, starting with the free ones.

7a. Groq (free, cloud, fastest to set up; recommended first)

1 Create a free account at **console.groq.com**, open **API Keys**, create a key, copy it (starts with `gsk_`).

2 In your breachmark folder create a file named `.env` (dot, then env). Easiest way, in PowerShell:

```
notepad .env
```

Click Yes to create it, paste these two lines with your key, save, close:

```
OPENAI_API_KEY=gsk_your_key_here
OPENAI_BASE_URL=https://api.groq.com/openai/v1
```

3 Check it works:

```
python -m breachmark doctor openai
```

You should see `openai: ok (API key: environment)` and a list of model names such as `llama-3.3-70b-versatile`, `qwen/qwen3-32b`, `openai/gpt-oss-120b`.

4 Start the app, go to **Runs** → **New run**, choose provider **OpenAI-compatible**, leave Server URL empty (it reads the `.env`), pick a model from the list.

Groq free limits: about 30 requests a minute and a tokens-per-minute cap. Set **Parallel requests = 1** and **Input limit = 20000** characters in the run form. If Groq throttles you, BreachMark waits and retries by itself.

7b. Google Gemini (free, cloud, biggest context window)

Get a key at **aistudio.google.com** (API keys). The free tier isn't available in the EU/UK, and Google may use free-tier prompts to improve its products. In `.env`:

```
OPENAI_API_KEY=your_gemini_key
OPENAI_BASE_URL=https://generativelanguage.googleapis.com/v1beta/openai
```

Model name: `gemini-2.5-flash` (or the newest Flash model AI Studio lists). Gemini's huge context means even the largest samples fit without truncation, so you can set Input limit to 0 (no limit).

Only one `OPENAI_BASE_URL` can be in `.env` at a time. To switch between Groq and Gemini, change the two lines, or type the Server URL directly in the run form instead (the form's Server URL overrides `.env`).

7c. Ollama (free, runs on your own PC, private)

Needs a reasonably modern PC: 8 GB RAM minimum for a 7B model, 16 GB comfortable; a GPU makes it 10× faster but isn't required.

- 1 Download and install Ollama from ollama.com/download (Windows). It runs in the background.
- 2 In PowerShell download a model (about 4.7 GB, one time):

```
ollama pull qwen2.5-coder:7b
```
- 3 Check: `python -m breachmark doctor ollama` should say `ollama: ok` and list the model.
- 4 In New run choose provider **Ollama (local)**. Keep Parallel requests at 1. Under Advanced, **Ollama context window** is 16384 tokens by default; raise it if you have RAM to spare, since some samples are long.

Speed without a GPU: roughly 30–90 seconds per question for a 7B model. A 40-sample test with one prompt (80 questions) is about an hour. Plan the full dataset only with a GPU or overnight.

7d. Paid APIs (DeepSeek, OpenAI, Anthropic)

Provider	.env lines	Provider in form / model example
DeepSeek	<code>OPENAI_API_KEY=sk-...</code> <code>OPENAI_BASE_URL=https://api.deepseek.com/v1</code>	OpenAI-compatible · <code>deepseek-chat</code>
OpenAI	<code>OPENAI_API_KEY=sk-...</code> (no base URL line)	OpenAI-compatible · <code>gpt-4o-mini</code>
Anthropic	<code>ANTHROPIC_API_KEY=sk-ant-...</code>	Anthropic · <code>claude-haiku-4-5</code>

Cost guide: a 40-sample run with two prompt styles is 160 questions, roughly 0.6M input tokens. That's a few cents on DeepSeek or GPT-4o-mini, about \$1 on Claude Haiku, and about \$3 on larger models.

8. Your first real experiment

This gives you a real result in under an hour and costs nothing with Groq. Do it in the web app.

- 1 **Runs** → **New run**. Provider: OpenAI-compatible (Groq). Model: `qwen/qwen3-32b` (or `llama-3.3-70b-versatile`).
- 2 **Prompt strategies**: keep **Baseline** and **Think & verify** ticked; untick the other two. (Four strategies is a bigger run; two shows the main effect.)
- 3 **Dataset slice**: Dataset **vulnsage** (matches the paper; choose All to mix in SVEN's Python code too), Sample limit **40**, Shuffle ticked with seed **1**. Keep both "Vulnerable code" and "Patched code" ticked.
- 4 **Options**: Run name "qwen3-32b normal", Parallel requests **1**, Input limit **20000**. Check the size line: it should say 40 samples, 160 model calls.

- 5 Click **Start run**. Watch the progress bar. If a question fails you'll see "failed" counts; ten failures in a row stop the run with the reason shown. When done, read the Results table and the confusion matrix.
- 6 **Now the interesting part:** create a second run, same everything, but tick **Blind mode** and name it "qwen3-32b blind". In blind mode the model isn't told which CWE to look for.
- 7 Go to **Compare**, tick both runs, click Compare selected. The difference between the normal and blind accuracy tells you how much of the model's "detection" came from the hint. That gap is a finding the original paper doesn't report.
- 8 Repeat steps 1–7 with a second model (the same seed guarantees the same 40 samples). Now you have a real comparison. Open a run and click **Report** to get a standalone HTML page you can share.

9. Reading the numbers

Column	Meaning	What to watch for
Accuracy	Correct answers over all answers. Unclear answers count as wrong.	50% is coin-flip level. The paper's best open models reached about 58% average, 72% at best.
95% CI	The range the true accuracy probably lies in, given how few samples you used.	With 40 samples the range is wide (about ± 15 points). Don't rank models on differences smaller than that.
Finds vuln (recall)	Of the vulnerable code shown, how often the model said YES.	High recall alone is cheap: a model that always says YES gets 100%.
Clears patch (specificity)	Of the patched (safe) code shown, how often the model said NO.	Low value means false alarms. Good models score well on both columns.
Pair correct	Samples where both the vulnerable and the patched answer were right.	The strictest measure. Often much lower than accuracy.
Ambiguous	Answers with no clear verdict.	Baseline prompts with chatty models cause this. Think & verify usually lowers it.
Latency / tokens	Speed and size of each answer.	Reasoning prompts cost 5–20× more output tokens than baseline.

The **confusion matrix** shows the same thing as a picture: top-left and bottom-right should be big (correct), top-right and bottom-left small. The **CWE heatmap** shows which bug types the model handles; green cells are above 50%, red below. With 40 samples most CWE cells have only 1–4 answers, so treat the heatmap as a hint until you run more.

10. Command-line use

Everything in the web app can also be done from PowerShell (with the venv active). Results show up in the web app too.

```
# check dataset, database and connections
python -m breachmark doctor

# the same first experiment as section 8, normal then blind
python -m breachmark run --provider openai --model qwen/qwen3-32b --strategies baseline
think_verify --limit 40 --concurrency 1 --max-input-chars 20000 --name "qwen3-32b
normal"
python -m breachmark run --provider openai --model qwen/qwen3-32b --strategies baseline
think_verify --limit 40 --concurrency 1 --max-input-chars 20000 --name "qwen3-32b
blind" --blind

# a local Ollama model, one strategy, 20 samples
python -m breachmark run --provider ollama --model qwen2.5-coder:7b --strategies
think_verify --limit 20

# start the web app on a different port
$env:BREACHMARK_PORT = "8010"; python -m breachmark
```

Press `Ctrl` + `C` during a command-line run to pause it; resume it from the Runs page.

11. Troubleshooting

Problem	Fix
"python is not recognized"	Python isn't on PATH. Re-run the installer, choose Modify, tick "Add Python to environment variables". Or use <code>py</code> instead of <code>python</code> in every command.
"Activate.ps1 cannot be loaded because running scripts is disabled"	Run <code>Set-ExecutionPolicy -Scope CurrentUser RemoteSigned</code> once. Or use Command Prompt instead of PowerShell and activate with <code>.venv\Scripts\activate.bat</code> .
"(.venv)" is missing from the prompt and pip installs the wrong place	Activate the environment again (section 4). Each new terminal window needs it.
"address already in use" / port 8000 busy	Something else uses port 8000. Start with <code>\$env:BREACHMARK_PORT = "8010"</code> ; <code>python -m breachmark</code> and open 127.0.0.1:8010.
Browser shows "Invalid host header"	You opened it with a name other than 127.0.0.1 or localhost. Use <code>http://127.0.0.1:8000</code> . To reach it from other devices, start with <code>\$env:BREACHMARK_HOST = "0.0.0.0"</code> .
Doctor says "Cannot connect to http://localhost:11434"	Ollama isn't running. Start it from the Start menu (it sits in the system tray) and try again.
"Model 'x' was not found on the Ollama server"	Run <code>ollama pull x</code> first, or pick a name from the list the run form shows.
"No API key found"	The <code>.env</code> file isn't in the folder you started from, or is named <code>.env.txt</code> . In File Explorer enable "File name extensions" to check. Or paste the key on the Settings page for this session.
Many "HTTP 429" failures with Groq or Gemini	Rate limit. BreachMark retries automatically; keep Parallel requests at 1 and lower the input limit. If a run stops as "failed", click Retry failed then Resume .
"HTTP 404 ... base URL ends in /v1"	The Server URL is wrong. Groq and DeepSeek need <code>/v1</code> at the end; Gemini needs <code>/v1beta/openai</code> .
Run shows "truncated" on many answers	Those samples were longer than the input limit and were shortened in the middle. Raise the limit (cloud models) or accept it; the flag is there so you know.
Everything is slow with Ollama	No GPU, or the model doesn't fit in RAM. Try a smaller model (<code>qwen2.5-coder:1.5b</code>) or use Groq.
Run says "paused: Interrupted when the app stopped"	Normal after closing the terminal mid-run. Open the run and click Resume ; nothing was lost.
Antivirus blocks or slows it	Add the breachmark folder to exclusions; SQLite writes a lot of small updates during a run.

12. Keys, updates and resetting

- **Keys:** keep them in `.env`, never in files you share or commit. The `.gitignore` already excludes `.env`. Keys typed on the Settings page vanish when the app stops. Rotate any key you have pasted into a chat or an email.
- **Updating BreachMark:** with Git, `git pull` in the folder. With the ZIP method, download again and copy your `.env` and `breachmark.db` into the new folder.
- **Where your data is:** one file, `breachmark.db`, in the folder you started from. Back it up by copying it. Delete it to start completely fresh (the dataset is re-imported on next start).
- **Adding more data:** Settings → Import a dataset. VulnSage and SVEN are built in. For PrimeVul, download its `*_paired.jsonl` files from github.com/DLVulDet/PrimeVul (Google Drive link in its README), enter the file path, choose format `primevul`, click Import. New samples get their own filter in the run form.
- **Exporting results:** on any run page, **Export CSV** (every answer) or **Report** (a self-contained HTML page). On Compare, Export CSV covers all selected runs.
- **Help:** the README and `docs/` folder in the repo, and `http://127.0.0.1:8000/api/docs` for the API while the app is running.

BreachMark is MIT-licensed, © 2026 Varshit Sharma. Datasets: VulnSage (Zibaeirad & Vieira, 2025, CC BY 4.0) and SVEN (He & Vechev, 2023, MIT).